

Deadline and Contention Aware GPU Memory Scheduling for Vision-Language-Action Models

Uday Arora

Course: CMPE 249, Intelligent Autonomous Systems, Fall 2026 **Team Member:** Uday Arora
Project Title: Deadline and Contention Aware GPU Memory Scheduling for Vision-Language-Action Models **GitHub Repository:** <https://github.com/uday101/vla-memory-scheduler> **Track:** Deployment Track

Abstract

Vision-Language-Action (VLA) models for autonomous driving, such as NVIDIA's open 10-billion-parameter Alpamayo, are too large to fit in the memory of commodity and automotive-grade GPUs. A peer-reviewed system, oom-free-alpamayo (IEEE RTCSA 2026), fixes the fit problem by streaming model layers between CPU and GPU on demand, but the specific set of layers it keeps GPU-resident is decided once, offline, purely to maximize throughput, with no notion of a control-loop deadline and no awareness of a second workload sharing the GPU. The result is an inference time far outside real-time bounds for autonomous driving, a gap this project does not aim to close outright, since no existing system in this space reaches true real-time. Instead, this project makes the residency decision itself runtime-adaptive, responding to a live deadline budget and to sensed GPU contention, and asks a narrower, answerable question: does that adaptivity measurably reduce the deadline-miss rate and degrade more gracefully under contention than the existing static policy.

Problem Formulation

Domain problem. NVIDIA's Alpamayo is an open 10-billion-parameter Vision-Language-Action model for autonomous driving. Its weights total 22GB, exceeding the 12 to 16GB VRAM typical of automotive and consumer GPUs. A peer-reviewed system, oom-free-alpamayo (IEEE RTCSA 2026), addresses this by streaming model layers between CPU and GPU memory on demand, with an offline profiling pass deciding which layers stay permanently GPU-resident to maximize throughput. This fits the model in memory, at reported inference times of 4.09 seconds (RTX 5070 Ti) and 15.46 seconds (RTX 3080 Ti), both configurations that fail without it. Alpamayo's own paper states that autonomous driving requires approximately 100ms end-to-end latency for real-time operation, a gap of 40 to 150 times. The residency policy that makes the model fit has no concept of a deadline and no awareness of contention from a second workload sharing the GPU. Five related systems were reviewed in full text; none combine deadline awareness, contention awareness, and intra-model memory-residency decisions for a VLA model. The closest, published in June 2026, is itself runtime-adaptive, but for a different problem: it load-balances a model that already fits across CPU and GPU, reacting to raw utilization and queue signals to migrate a small, fixed set of boundary layers. This project instead fits a model that does not fit in GPU memory at all, reacting to an explicit deadline budget and sensed GPU contention across the full residency decision, not a bounded migration zone.

Scope of the claim. This project does not aim to make Alpamayo meet the 100ms target. Even the newest published competitor, with substantially more engineering investment, reaches only 306 to 408ms on real driving VLA models. The contribution is relative: whether a deadline and contention aware residency policy measurably reduces the deadline-miss rate and degrades more gracefully under contention than the existing static policy.

Input and output. Input: Alpamayo’s normal multi-camera and trajectory-history input, plus two additional signals not used by the current system: a per-call deadline budget, and a live GPU contention estimate obtained via NVML or DCGM, or a known injected level during controlled experiments. Output: the model’s normal trajectory and reasoning output, plus a per-call record of which layers were kept resident and the resulting latency, for analysis.

Target metrics and success criteria, fixed in advance.

Primary metric: deadline-miss rate, the fraction of inference calls exceeding a stated latency budget, compared between the adaptive policy and the unmodified static baseline under injected GPU contention.

Secondary metric: the shape of the latency degradation curve as contention increases.

Statistical test: a paired Wilcoxon signed-rank test, effect size as matched-pairs rank-biserial correlation, at least eight repetitions per scenario, alpha of 0.05.

Contention scenarios: a steady compute-bound load using a synthetic contention generator modeled on Kutukcu et al., and a bursty load injected at randomized intervals. Both run as an independent process sharing the GPU through CUDA MPS.

Null-result framing: a statistically undetected improvement is reported as exactly that, a valid and reportable outcome.

Proposed Technical Approach

Foundation. The project forks aveeslab/oom-free-alpamayo, whose codebase is small and cleanly modular, with a documented adapter interface and a single function implementing the GPU-Resident Layer Decision Policy. The chosen model is nvidia/Alpamayo-R1-10B, open and ungated, licensed under OpenMDW-1.1, an open academic license permitting free non-commercial use. Early implementation verifies that this function’s isolation holds in practice and that CUDA MPS supports two concurrent processes on the target GPU.

High-level architecture.

Baseline: the unmodified upstream static residency policy, run exactly as released.

Adaptive policy: replaces the one-time offline profiling decision with a policy that re-evaluates residency at runtime using the remaining deadline budget and a live GPU-occupancy signal.

Contention injection: a synthetic dummy-workload contention generator and a bursty-arrival generator, run as an independent process sharing the GPU through CUDA MPS.

Evaluation: both policies run under both contention scenarios, at least eight repetitions each, differing only in the residency policy.

Data sources. The model and codebase require no special access beyond the license above. Representative driving-scene input frames come from NVIDIA's PhysicalAI-Autonomous-Vehicles dataset, gated by an automatic license agreement rather than manual review, or Jonas-all/Alpamayo-CARLA-bridge, an independently maintained CARLA-simulator integration.

Intended technical modifications. A runtime-adaptive layer-residency policy replacing the static one; a deadline and contention sensing layer feeding that policy; a contention-injection harness; and an evaluation pipeline comparing deadline-miss rate and degradation-curve shape rather than raw throughput alone.

Device Available and Maintainer

Device available: NVIDIA RTX 5050 (desktop) for code development; model loading, inference, and contention experiments run on a rented A100-class GPU, since Alpamayo-R1-10B's 22GB footprint exceeds the local card's memory. Maintainer: Uday Arora; Claude Code access is requested to support implementation throughout the semester.